

Week 4 – Text Classification with Naïve Bayes

DS3010: Introduction to Machine Learning Lab

Timing: 02:00 PM to 04:45 PM

Max Marks: 10

Instructions

1. Submit one `.ipynb` file containing all answers. The name should be `[student_name]_[rollno]_[lab].ipynb`
2. Write the questions in separate text blocks before the answers.
3. Outputs for all sub-questions should be given and the code should be executable.
4. Write justifications for your choices where needed.
5. Ensure that all plots include clear labels and legends for better interpretation.
6. The Naïve Bayes classifier must be implemented **from scratch**. You may use `numpy` and `matplotlib` for computation and plotting, but the use of machine learning libraries (such as `scikit-learn`) for the classifier is strictly prohibited.
7. Use of generative AI tools (such as ChatGPT, Gemini, etc.) is strictly prohibited. Any submission found to contain AI-generated or plagiarized content will receive a score of zero, and disciplinary action.

Dataset

In this lab session, you will explore **Text Classification** using the **20 Newsgroups** dataset. Visit <http://qwone.com/~jason/20Newsgroups/> for a description of the dataset. You must use the **bydate** version, available at: https://drive.google.com/file/d/1Sv1-tOMryapEbyRonfjAiLw_sEI1LSdw/view?usp=sharing

The dataset contains approximately 18,800 documents organized into 20 classes and is pre-split into training and test sets. Use the provided train and test splits directly; do not reshuffle them.

Setup: accessing the dataset

Download the `20news-bydate.tar.gz` archive from the link above and extract it on your machine. After extraction you will have two folders, one for training and one for testing, each containing 20 sub-folders (one per class):

```
20news-bydate-train/<class_name>/<document_id>
20news-bydate-test/<class_name>/<document_id>
```

In your notebook, point the code at the folder where you extracted the archive (edit the path to match your own machine), and verify that the 20 class folders are visible:

```
import os

# point this at wherever you extracted the archive on your machine
DATA_ROOT = "/home/user/Downloads/20news-bydate" # <- your actual path
TRAIN_DIR = os.path.join(DATA_ROOT, "20news-bydate-train")
TEST_DIR = os.path.join(DATA_ROOT, "20news-bydate-test")

print(sorted(os.listdir(TRAIN_DIR))) # should list the 20 class folders
```

Preprocessing

Before training, convert each raw document into a list of word tokens. You are expected to write this yourself, but the following pointers describe the approach so that everyone preprocesses the data consistently. You only need basic Python string methods — no special libraries are required.

- **Strip the header.** Each file begins with an email-style header (lines such as `From:`, `Subject:`, `Organization:`) followed by a blank line and then the message body. Keep only the body. Since a blank line appears in the text as `"\n\n"`, you can use `text.split("\n\n", 1)` to break the file into header and body, and keep the second part.
- **Lowercase and split.** Convert the text to lowercase with `text.lower()` and split it into words with `text.split()` (which splits on spaces and newlines).
- **Clean each word.** Remove surrounding punctuation with `word.strip(".,!()?[]:;")` (add any other symbols you see). Keep a word only if it is alphabetic and longer than one character — `word.isalpha()` and `len(word) > 1` are useful here. This discards numbers and punctuation-only tokens.
- **Stopwords.** For the stopwords-removal part, read the Mallet `en.txt` list into a Python `set` (one word per line) and drop any token that is in it. Download the file and place it in your data folder. Checking membership against a `set` is fast.

A convenient design is to write a single `tokenize(text)` function that performs all the steps above (and optionally removes stopwords), then reuse it for every document in both the training and test sets.

Evaluation. Compute the accuracy and the confusion matrix, without a machine learning library. The accuracy is simply the fraction of test documents whose predicted class equals the true class. The confusion matrix is a 20×20 table whose entry (i, j) counts how many documents of true class i were predicted as class j ; build it by counting over the test set, and display it with `matplotlib` (for example, `plt.imshow`).

Naïve Bayes Classifier

1. Baseline classifier

- (a) Load the training and test documents. For each document, remove the email-style header (the lines before the first blank line) and tokenize the remaining text into lowercase words.
- (b) Implement a multinomial Naïve Bayes classifier from scratch. Estimate the class priors and the smoothed word likelihoods $P(\text{word} \mid \text{class})$ from the **training set only**. Use additive (Laplace) smoothing with $\alpha = 1$. Perform all computations in log-space.
- (c) Classify the test documents and report the overall classification accuracy.
- (d) Compute and plot the confusion matrix for the 20 classes. Ensure the axes are clearly labelled.

2. Effect of stopwords removal

- (a) Remove the stopwords from each document using the stopwords list provided at: <https://github.com/mimno/Mallet/blob/master/stoplists/en.txt>

- (b) Train another Naïve Bayes classifier ($\alpha = 1$) on the stopword-removed documents and report its accuracy on the test set.
- (c) Plot the confusion matrix for this classifier.

3. Effect of the smoothing parameter

- (a) Experiment with different values of the smoothing parameter α (for example, 0.001, 0.01, 0.1, 1, 10), for both the original and the stopword-removed versions.
- (b) Report the test accuracy for each value of α in a table.

Analysis

- 4. Print the classification performance (accuracy/F1) after removing stopwords, and after changing the smoothing parameter.
- 5. Print the ten most probable words for each class from the trained Naïve Bayes classifier.
- 6. Print the confusion matrix for the classifier.

All the best for your lab! Happy coding!